

and three straight sides, we get a potentially infinite encyclopedia of unique centers—Leibniz would be so pleased.

We can write a function that finds the circumcenter and *circumradius* (the radius of the circumcircle) for any given triangle. This function relies on conversion to complex numbers. It takes a triangle as its input and returns a center and a radius as its output:

```
def triangle_to_circumcenter(triangle):
    x,y,z = complex(triangle[0][0],triangle[0][1]), complex(triangle[1]
[0],triangle[1][1]), \    complex(triangle[2][0],triangle[2][1])
    w = z - x
    w /= y - x
    c = (x-y) * (w-abs(w)**2)/2j/w.imag - x
    radius = abs(c + x)
    return((0 - c.real,0 - c.imag),radius)
```

The specific details of how this function calculates the center and radius are complex. We won't dwell on it here, but I encourage you to walk through the code on your own, if you'd like.

Increasing Our Plotting Capabilities

Now that we can find a circumcenter and a circumradius for every triangle, let's improve our `plot_triangle()` function so it can plot everything. [Listing 7-2](#) shows the new function.

```
def plot_triangle(triangles,centers,radii,thename):
    fig, ax = pl.subplots()
    ax.set_xlim([0,1])
    ax.set_ylim([0,1])
    for i in range(0,len(triangles)):
        triangle = triangles[i]
        center = centers[i]
        radius = radii[i]
        itin = [0,1,2,0]
        thelines = genlines(triangle,itin)
        xs = [triangle[0][0],triangle[1][0],triangle[2][0]]
        ys = [triangle[0][1],triangle[1][1],triangle[2][1]]

        lc = mc.LineCollection(genlines(triangle,itin), linewidths = 2)

    ax.add_collection(lc)
```